

Rapport des Transformations ETL

Pipeline Python — 9 regles commandes | 5 regles clients | 3 regles produits

Resume global

Source	Lignes brutes	Lignes nettes	Rejetes	Taux rejet
commandes_mexora.csv	51 500	49 189	2 311	4.5%
clients_mexora.csv	3 000	2 878	122	4.1%
produits_mexora.json	20	20	0	0%
regions_maroc.csv	15	15	0	0% (propre)

Pipeline complet execute en 3.6 secondes — CA total TTC genere (livrees) : 1 462 168 399 MAD

Transformations — Commandes (clean_commandes.py)

R1 Doublons id_commande

Suppression des doublons sur id_commande. On conserve la DERNIERE occurrence (hypothese : la saisie la plus recente est la correction).

```
df.drop_duplicates(subset=['id_commande'], keep='last')
```

Impact : 51 500 -> 50 000 | 1 500 doublons supprimees (2.9%)

R2 Standardisation des dates

Harmonisation vers YYYY-MM-DD. Trois formats sources detectes : ISO (2024-11-15), francais (15/11/2024), anglais textuel (Nov 15 2024).

```
pd.to_datetime(format='mixed', dayfirst=True, errors='coerce')
```

Impact : 0 dates invalides | Tous les formats parses avec format='mixed'

R3 Harmonisation des villes

Normalisation via le referentiel regions_maroc.csv (seule source de verite). Ex : tanger/TNG/TANGER/Tnja -> Tanger. Villes non reconnues -> 'Non renseignee'.

```
df['ville'].str.lower().map(mapping_villes).fillna('Non renseignee')
```

Impact : 0 villes non reconnues | Couverture totale du referentiel

R4 Standardisation des statuts

8 variantes sources mapées vers 4 valeurs cibles : livre | annule | en_cours | retourne. Valeurs non reconnues -> 'inconnu'.

```
mapping = {'DONE':'livre', 'KO':'annule', 'OK':'en_cours', ...}
```

Impact : 0 statuts inconnus | Couverture totale du mapping

R5 Quantites invalides

Suppression des lignes avec quantité ≤ 0. Une quantité négative est une erreur de saisie opérateur — sans valeur réelle connue, la ligne ne peut pas alimenter les KPIs.

```
df[df['quantite'].astype(float) > 0]
```

Impact : 50 000 -> 49 450 | 550 lignes supprimées

R6 Commandes test prix=0

Suppression des commandes à prix nul. Mexora crée des commandes test à 0 MAD pour tester le système de paiement. Elles fausseraient le CA.

```
df[df['prix_unitaire'].astype(float) > 0]
```

Impact : 49 450 -> 49 189 | 261 commandes test supprimées

R7 Livreurs manquants

7% des commandes n'ont pas de livreur renseigné (sous-traitance non tracée). On remplace par id=-1 et on crée un livreur virtuel 'Inconnu' dans dim_livreur pour ne pas casser l'intégrité référentielle.

```
df['id_livreur'].replace('', '-1').fillna('-1')
```

Impact : 3 543 valeurs manquantes remplacées par id = -1

R8 Calcul montants HT/TTC

TVA Maroc standard = 20%. Le prix_unitaire source est HT. On calcule les deux versions : HT pour analyses fournisseurs, TTC pour CA client.

```
montant_ht = quantite * prix_unitaire | montant_ttc = montant_ht * 1.20
```

Impact : 49 189 lignes enrichies | Aucune suppression

R9 Delai de livraison

Calcul en jours pour les commandes livrées et retournées. Les délais négatifs (erreur saisie) et > 60 jours (aberrant) sont nullifiés.

```
(date_livraison - date_commande).dt.days | null si < 0 ou > 60
```

Impact : 34 365 délais calculés | Délais aberrants nullifiés

Transformations — Clients (clean_clients.py)

R1 Deduplication email

Meme email = meme personne. Une migration passee a cree des doublons (meme email, id_client different). On conserve l'inscription la plus recente.

```
sort_values('date_inscription').drop_duplicates(subset=['email_norm'], keep='last')
```

Impact : 3 000 -> 2 878 | 122 doublons supprimees

R2 Standardisation du sexe

3 systemes sources avec 3 conventions differentes : m/f (A), 1/0 (B), Homme/Femme (C). Cible unifiee : m / f / inconnu.

```
mapping = {'1':'m', '0':'f', 'Homme':'m', 'Femme':'f', ...}
```

Impact : 0 valeurs inconnu | Couverture totale

R3 Validation dates naissance

Plage acceptee : 16 a 100 ans (legal e-commerce Maroc). Hors plage -> nullifie. Le client est conserve, sans date de naissance fiable.

```
age < 16 | age > 100 -> date_naissance = None
```

Impact : 246 dates de naissance nullifiees

R4 Validation format email

Email sans @ ou sans domaine -> nullifie. Le client reste dans le DWH mais est exclu des campagnes email marketing.

```
regex = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
```

Impact : 187 emails nullifies

R5 Harmonisation villes clients

Meme logique que R3 des commandes — referentiel officiel comme source de verite pour normaliser les noms de villes.

```
df['ville'].str.lower().map(mapping_villes).fillna('Non renseignee')
```

Impact : 0 villes non reconnues

Segmentation Gold / Silver / Bronze

Calculee depuis les commandes livrees des 12 derniers mois.

Gold >= 15 000 MAD | Silver >= 5 000 MAD | Bronze < 5 000 MAD

Resultat : Gold : 2 784 clients | Silver : 75 | Bronze : 38

Transformations — Produits (clean_produits.py)

R1 Standardisation categories

Casse incohérente dans le fichier source : électronique / ELECTRONIQUE / Electronique. Normalisation en Title Case pour toutes les catégories.

```
df['categorie'].str.strip().str.title()
```

Impact : 20 produits normalisés

R2 Prix catalogue nuls

Certains anciens produits n'ont pas de prix catalogue. On remplace null par -1.00 (convention : prix inconnu, distinct de 0.00 qui serait trompeur).

```
df['prix_catalogue'].fillna(-1.0)
```

Impact : 0 remplacements dans ce run (aucun prix null genre)

R3 Produits inactifs SCD

Les produits marques actif=false ont des commandes historiques. On les CONSERVE pour garantir la cohérence du SCD Type 2.

```
df['actif'] = df['actif'].astype(bool) # conserve, flag False
```

Impact : 3 produits inactifs conservés

Logs complets disponibles dans logs/etl_YYYYMMDD_HHMMSS.log après exécution de main.py